Demand-Driven Learning Platform Specification

A Comprehensive Engineering, Product, Monetization, and Multi-Channel Fulfillment Implementation Roadmap for On-Demand Digital Skill Video Tutorials.

1. Executive Summary & Value Proposition

This document formalizes the full software architecture, business rules, and step-by-step implementation plan for a demand-driven educational engine. The application shifts the traditional content creator model from *speculative recording* (guessing what viewers want) to *guaranteed audience fulfillment* (recording tutorials that users explicitly request, back with upvotes, or commission via direct private payment).

DEMAND CONVERSION
100% Validated
Zero recording before verified community demand

MONETIZATION ENGINE
Hybrid Tier
Free Community Upvoting + Private Paid Escrow

CONTENT FLYWHEEL
Auto SEO (ISR)
Public tickets convert to Google-indexed tutorials

2. Cross-Disciplinary Architecture & Strategic Optimization

To ensure high operational efficiency and eliminate creator burnout, seven specialized engineering disciplines have been synthesized into this platform design:

> **1. Scalability Engineering (Deduplication Engine):** Standard ticketing creates duplicate requests (e.g., 50 distinct tickets for "React Auth"). We implement fuzzy semantic search upon title entry to route learners to upvote existing public tickets. One video fulfills hundreds of learners simultaneously.

> **2. Hybrid Revenue Model (Free vs. Private Commission):** Community requests are public and free, fueling YouTube subscribers and public SEO. High-touch, confidential, or urgent requests (debugging proprietary company setups, code reviews) operate through an upfront checkout workflow with guaranteed delivery windows.

> **3. Multi-Channel Notification Orchestration:** Email notification open rates decay rapidly. The delivery engine combines transactional email (Postmark/Resend), push web notifications, and transactional SMS/WhatsApp webhooks so users access their custom lesson within seconds of publishing.

3. High-Level Data Model & Architecture Matrix

ENTITY / DOMAIN	KEY ATTRIBUTES	STRATEGIC RESPONSIBILITY & INTEGRITY RULES
Users & Roles	`id, email, role, phone, notify_prefs`	Enforces RBAC (`learner`, `admin_creator`). Stores WhatsApp/email opt-in preferences.
LearningRequests	`id, type, title, context, status, upvotes`	Central ticket entity. Type enum: `PUBLIC_FREE` vs `PRIVATE_PAID`. Status machine enforced at DB level.
RequestOrders	`id, request_id, amount, gateway_ref, status`	Manages payment state (`pending`, `paid`, `refunded`) for private bespoke tickets via Webhook reconciliation.

ENTITY / DOMAIN	KEY ATTRIBUTES	STRATEGIC RESPONSIBILITY & INTEGRITY RULES
Fulfillments	`id, request_id, yt_video_id, notes, repo_url`	Binds YouTube metadata, duration, resources, and private tokenized video players to closed tickets.

4. Complete Ticket State Lifecycle Machine

```
[PUBLIC REQUEST]:  Submitted ──▶ Under Community Review ──▶ Backlog Scheduled ──▶ In Production ──▶ Fulfilled (Public Watch Page & Alert)
[PRIVATE COMMISSION]: Draft ──▶ Checkout / Escrow ──▶ Payment Verified ──▶ In Production (SLA) ──▶ Fulfilled (Protected Player & Delivery Alert)
```

5. Ultra-Granular Implementation Roadmap (Phase 01–04)

The following master breakdown details current status, technical connections, objectives, and prerequisites for all implementation phases.

STEP ID	STATUS	ACTION ITEM & SCOPE	WHY IT'S DONE & OBJECTIVE	SYSTEM CONNECTIONS & TOOLING
PH-01.1	COMPLETED	System Architecture & Product Strategy	Eliminate creator bottlenecks; establish the public/private monetization divide.	System blueprint, functional specification matrix.
PH-01.2	NEXT UP	Relational Schema & Database Provisioning	Establish atomic relational tables, foreign key constraints, indexes on upvotes and statuses.	PostgreSQL (Neon / Supabase), Prisma ORM, migrations engine.
PH-01.3	PENDING	Authentication & RBAC Enforcement	Secure learner data and protect creator-only fulfillment mutation endpoints.	NextAuth.js / Supabase Auth, Google OAuth, GitHub provider.
PH-02.1	PENDING	Fuzzy Deduplication & Search API	Prevent redundant tickets; drive community users to back existing topics.	PostgreSQL pg_trgm extension, full-text vector indexing.
PH-02.2	PENDING	Ticket Submission Flow (Form Engine)	Capture granular learner skill requirements, error logs, and context level.	React Hook Form, Zod validation schemas, Tailwind UI components.
PH-02.3	PENDING	Secure Attachment Storage Service	Permit upload of code samples, architecture diagrams, and error screenshots.	Cloudflare R2 / AWS S3 presigned PUT URLs, MIME type security guard.

STEP ID	STATUS	ACTION ITEM & SCOPE	WHY IT'S DONE & OBJECTIVE	SYSTEM CONNECTIONS & TOOLING
PH-03.1	PENDING	Upvoting & Watchlist Subscription Engine	Prioritize video demand and bind audience to launch notification queues.	Next.js Server Actions, optimistic UI updates, atomic increment triggers.
PH-03.2	PENDING	Public Community Demand Leaderboard	Expose most requested digital skills with filtering by category and status.	React Server Components (RSC), URL search params state caching.
PH-04.1	PENDING	Payment Gateway Integration (Stripe & Paystack)	Accept payments for private bespoke requests and priority SLA delivery.	Stripe Checkout API, Paystack API, dynamic session initialization.
PH-04.2	PENDING	Idempotent Webhook Handler & State Transition	Safely unlock production queues only upon cryptographic payment verification.	Next.js Route Handlers (Edge), raw body signing verification, Postgres transactions.

6. Ultra-Granular Implementation Roadmap (Phase 05–08)

STEP ID	STATUS	ACTION ITEM & SCOPE	WHY IT'S DONE & OBJECTIVE	SYSTEM CONNECTIONS & TOOLING
PH-05.1	PENDING	Creator Kanban Fulfillment Dashboard	Empower the creator to view top-voted topics, paid SLAs, and pending recordings.	DnD-kit / TanStack Table, creator-authenticated layout route.
PH-05.2	PENDING	YouTube URL Parser & API Metadata Ingest	Validate YouTube links, extract video duration, HD thumbnail, and embed parameters.	Google YouTube Data API v3, regex embed sanitization engine.
PH-05.3	PENDING	Single-Click Resolution Dispatcher	Creator attaches YouTube URL, repo links, and triggers global notifications instantly.	Background queue workers (Upstash QStash / BullMQ).
PH-06.1	PENDING	Transactional Email Notification Dispatch	Deliver branded, high-deliverability video-ready notifications with direct lesson links.	Resend / Postmark, React Email templating engine.
PH-06.2	PENDING	Instant SMS & WhatsApp Delivery Integration	Bypass clogged email inboxes for instant learning gratification.	Twilio / Termii WhatsApp Business API webhook integrations.
PH-06.3	PENDING	In-App Real-time Notification Bell	Give active platform users real-time badges when their tickets move through stages.	Supabase Realtime / Server-Sent Events (SSE) broadcast channel.

STEP ID	STATUS	ACTION ITEM & SCOPE	WHY IT'S DONE & OBJECTIVE	SYSTEM CONNECTIONS & TOOLING
PH-07.1	PENDING	Static Public Watch Pages (ISR)	Turn every solved ticket into an evergreen, Google-indexed tutorial hub.	Next.js Incremental Static Regeneration (`generateStaticParams`).
PH-07.2	PENDING	Schema.org VideoObject & Educational JSON-LD	Capture Google Video rich snippets, carousels, and search badges automatically.	Structured Data (Schema.org) injection via Next.js Metadata API.
PH-07.3	PENDING	Tokenized Private Video Viewer Engine	Prevent unauthorized sharing of bespoke paid tutorials to non-paying users.	Session verification guards, unlisted iframe wrappers, anti-indexing headers.
PH-08.1	PENDING	Rate Limiting, Abuse Prevention & WAF	Prevent spam tickets, denial-of-service on upvote counters, and bot attacks.	Upstash Redis sliding-window algorithm, Cloudflare Turnstile CAPTCHA.
PH-08.2	PENDING	Production CI/CD & Performance Hardening	Zero-downtime deployment, sub-second TTFB, automated branch preview testing.	Vercel Edge Network / AWS Amplify, GitHub Actions CI/CD pipeline.

7. Production Database Schema Definition (Prisma)

The core structural schema enforcing user relations, payment transactions, and video fulfillment integrity:

```
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

generator client {
  provider = "prisma-client-js"
}

enum UserRole {
  LEARNER
  CREATOR_ADMIN
}

enum RequestTier {
  PUBLIC_FREE
  PRIVATE_PAID
}

enum RequestStatus {
  SUBMITTED
  UNDER_REVIEW
  APPROVED_BACKLOG
  IN_PRODUCTION
  FULFILLED
  DECLINED
}

enum PaymentStatus {
  PENDING
  SUCCESSFUL
  FAILED
  REFUNDED
}

model User {
  id              String           @id @default(uuid())
  email           String           @unique
  name            String?
  role            UserRole         @default(LEARNER)
  phoneNumber     String?
  notifyViaEmail  Boolean          @default(true)
  notifyViaSMS    Boolean          @default(false)
  createdAt       DateTime         @default(now())
  requests        LearningRequest[]
  upvotes         RequestUpvote[]
  orders          PaymentOrder[]
}

model LearningRequest {
  id              String           @id @default(uuid())
  userId          String
  user            User             @relation(fields: [userId], references: [id])
  tier            RequestTier      @default(PUBLIC_FREE)
  title           String
  slug            String           @unique
  description     String
  skillCategory   String
  skillLevel      String           // Beginner, Intermediate, Advanced
  status          RequestStatus    @default(SUBMITTED)
  upvoteCount     Int              @default(0)
```

8. Fulfillment Video Ingestion & Delivery Routine

When the creator pastes a YouTube URL into the admin dashboard, the system executes an automated workflow to ensure seamless delivery and security:

> **Step 1: Ingestion & Verification:** The backend verifies the link against YouTube's oEmbed endpoint, extracts the canonical `videoId`, validates duration, and generates static thumbnails.
> **Step 2: Access Rule Configuration:** If the ticket is `PUBLIC_FREE`, the status flips to `FULFILLED`, generating a public page at `/tutorials/[slug]` with Schema.org video tags. If `PRIVATE_PAID`, the video is flagged as unlisted, and access is tokenized to the original purchaser's authenticated dashboard.
> **Step 3: Multi-Channel Event Dispatch:** An asynchronous job queue notifies the author and all backers via email, in-app notification, and optional WhatsApp alert with the direct link.